

Flow Control

Mao

Outline



- **The bool Data Type**
- **Logical Operators**
- **getchar() and putchar()**
- **Selection**
- **Switch Statements**
- **Conditional Expressions**
- **Operator Precedence Chart**

Outline



- **Loops**
 - **The while Loop**
 - **The do-while Loop**
 - **The for Loop**
 - **Nested Loops**
- **Keywords break and continue**

The bool Data Type

The **bool data type** declares a variable with the value either **true** or **false**.

<i>Operator</i>	<i>Mathematics Symbol</i>	<i>Name</i>	<i>Example (radius is 5)</i>	<i>Result</i>
<	<	less than	<code>radius < 0</code>	<code>false</code>
<=	≤	less than or equal to	<code>radius <= 0</code>	<code>false</code>
>	>	greater than	<code>radius > 0</code>	<code>true</code>
>=	≥	greater than or equal to	<code>radius >= 0</code>	<code>true</code>
==	=	equal to	<code>radius == 0</code>	<code>false</code>
!=	≠	not equal to	<code>radius != 0</code>	<code>true</code>

The equality testing operator is **two equal signs (==)**, not a single equal sign (=). The latter symbol is used for assignment.

The bool Data Type

A variable that holds a Boolean value is known as a Boolean variable. The **bool data type** is used **to declare Boolean variables**. For example,

```
bool lightsOn = true;
```

Internally, C uses **1 to represent true and 0 for false**.

If you **display a bool value to the console**, **1** is displayed if the value is true and **0** if it is false.

Logical Operators:

Logical operators normally take relational expressions as operands. The **!** operator takes one operand. The rest take two—one to the left, one to the right.

Operator	Meaning
& &	and
	or
!	not

Logical Expressions:

expression1 && expression2 is true if and only if both expressions are true.

expression1 || expression2 is true if either one or both expressions are true.

!expression is true if the expression is false, and vice versa.

AND Truth Table

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

OR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

XOR Truth Table

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

NOT Truth Table

A	B
0	1
1	0

Operator Precedence Chart

Operator precedence and associativity determine the order in which operators are evaluated.

Precedence	Operator
	<code>var++</code> and <code>var--</code> (Postfix)
	<code>+</code> , <code>-</code> (Unary plus and minus), <code>++var</code> and <code>--var</code> (Prefix)
	<code>static_cast<type>(v)</code> , (type) (Casting)
	<code>!</code> (Not)
	<code>*</code> , <code>/</code> , <code>%</code> (Multiplication, division, and remainder)
	<code>+</code> , <code>-</code> (Binary addition and subtraction)
	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> (Relational)
	<code>==</code> , <code>!=</code> (Equality)
	<code>&&</code> (AND)
	<code> </code> (OR)
	<code>=</code> , <code>+=</code> , <code>-=</code> , <code>*=</code> , <code>/=</code> , <code>%=</code> (Assignment operator)

Operator Precedence Chart



What is the value of the following expression?

```
int i = 3 + 4 * 4 > 5 * (4 + 3) - 1 && (4 - 3 > 5);  
printf("%d", i);
```

It's better to use parentheses to make it more readable.

```
int i = (3 + 4 * 4 > 5 * (4 + 3) - 1) && (4 - 3 > 5);  
printf("%d", i);
```

getchar() and putchar()

The `getchar()` function takes no arguments, and it returns the next character from input. For example, the following statement reads the next input character and assigns its value to the

variable `ch` :

```
ch = getchar();
```

This statement has the same effect as the following statement:

```
scanf("%c", &ch);
```

getchar() and putchar()

The `putchar()` function prints its argument. For example, the next statement prints as a character the value previously assigned to `ch` :

```
putchar(ch);
```

This statement has the same effect as the following:

```
printf("%c", ch);
```

To test different character, `#include <ctype.h>` can be added into a C program.

Table 7.1 The `ctype.h` Character-Testing Functions

Name	True If the Argument Is
<code>isalnum()</code>	Alphanumeric (alphabetic or numeric)
<code>isalpha()</code>	Alphabetic
<code>isblank()</code>	A standard blank character (space, horizontal tab, or newline) or any additional locale-specific character so specified
<code>iscntrl()</code>	A control character, such as Ctrl+B
<code>isdigit()</code>	A digit
<code>isgraph()</code>	Any printing character other than a space
<code>islower()</code>	A lowercase character
<code>isprint()</code>	A printing character
<code>ispunct()</code>	A punctuation character (any printing character other than a space or an alphanumeric character)
<code>isspace()</code>	A whitespace character (a space, newline, formfeed, carriage return, vertical tab, horizontal tab, or, possibly, other locale-defined character)
<code>isupper()</code>	An uppercase character
<code>isxdigit()</code>	A hexadecimal-digit character
<code>tolower()</code>	If the argument is an uppercase character, this function returns the lowercase version; otherwise, it just returns the original argument.
<code>toupper()</code>	If the argument is a lowercase character, this function returns the uppercase version; otherwise, it just returns the original argument.

Selection

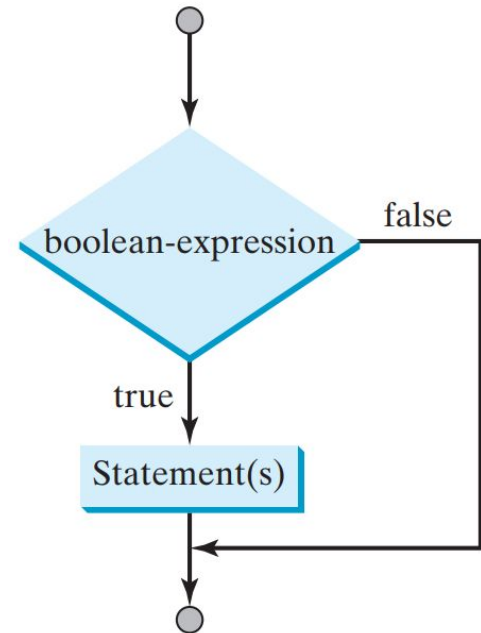


C provides several types of selection statements: one-way if statements, two-way if-else statements, nested if statements, switch statements, and conditional expressions.

Selection–One-way if Statements

A one-way if statement executes an action if and only if the condition is true. The syntax for a one-way if statement is shown here:

```
if (boolean-expression)
{
    statement(s);
}
```



Selection–Nested if and Multi-Way if-else Statements

The braces can be omitted if they **enclose a single statement**. For example, the following statements are equivalent.

```
if (i > 0)
{
    printf("i is positive" );
}
```

```
if (i > 0)
    printf("i is positive" );
```

Selection–One-way if Statements

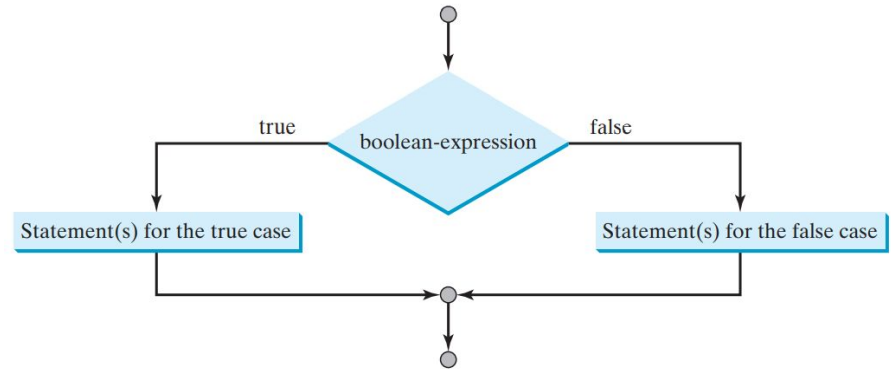
The simple form of an if statement gives you the choice of executing a statement (possibly compound) or skipping it.

```
if (all_days != 0)
    printf("%d days total: %.1f%% were below freezing.\n",
        all_days, 100.0 * (float) cold_days / all_days);
if (all_days == 0)
    printf("No data entered!\n");
```

Selection–Two-Way if-else Statements

A two-way if-else statement specifies different actions, depending on whether the condition is true or false. Here is the syntax for a two-way if-else statement:

```
if (boolean-expression)
{
    statement(s)-for-the-true-case;
}
else
{
    statement(s)-for-the-false-case;
}
```



Selection–One-way if Statements

The previous program finds that `all_days` is not equal to 0 , it should know that days must be 0 without retesting, and it does. With `if else` , you can take advantage of that knowledge by rewriting the fragment this way:

```
if (all_days!= 0)
    printf("%d days total: %.1f%% were below freezing.\n",
        all_days, 100.0 * (float) cold_days / all_days);
else
    printf("No data entered!\n");
```

Selection–Nested if and Multi-Way if-else Statements

An if statement can be inside another if statement to form a nested if statement. The inner if statement can contain another if statement; in fact, **there is no limit to the depth of the nesting**. For example, the following is a nested if statement:

```
if (i > k)
{
    if (j > k)
        printf("i and j are greater than k" );
}
else
    printf("i is less than or equal to k");
```

Selection–Nested if and Multi-Way if-else Statements

If there are multiple branches, **else if** can be used as follows:

```
if (condition1) {  
    // block of code to be executed if condition1 is true  
} else if (condition2) {  
    // block of code to be executed if the condition1 is false and condition2 is true  
} else if (condition3) {  
    // block of code to be executed if the condition1 and 2 are false and condition3 is true  
} else {  
    // block of code to be executed if all the previous conditions are false  
}
```

Case Study: Computing Taxes

Personal income tax brackets and rates - 2021 tax year (BC, Canada)

Taxable Income - 2021 Brackets	Tax Rate
\$0 to \$42,184	5.06%
\$42,184.01 to \$84,369	7.70%
\$84,369.01 to \$96,866	10.50%
\$96,866.01 to \$117,623	12.29%
\$117,623.01 to \$159,483	14.70%
\$159,483.01 to \$222,420	16.80%
Over \$222,420	20.5%

[Personal Income Tax Rates - Province of British Columbia \(gov.bc.ca\)](https://www.gov.bc.ca/taxrates/)

Switch Statements

A switch statement executes statements **based on the value of a variable or an expression**. Here is the full syntax for the switch statement:

```
switch (switch-expression)
{
    case value1:    statement(s)1;
                  break;
    case value2:    statement(s)2;
                  break;
    ...
    case valueN:    statement(s)N;
                  break;
    default:        statement(s)-for-default;
}
```

Switch Statements

The switch statement observes the following rules:

- The **switch-expression must yield an integral value** and always be **enclosed in parentheses**.
- The **value1, . . . , and valueN** are **integral constant expressions**, meaning that **they cannot contain variables**, such as $1 + x$. These values are **integers and cannot be floating-point values**.
- When the value in a case statement matches the value of the switch-expression, the statements starting from this case are **executed until either a break statement or the end of the switch statement is reached**.
- The default case, which is optional, can be used to perform actions when **none of the specified cases matches the switch-expression**.
- The keyword **break is optional**. The break statement immediately ends the switch statement.

Conditional Expressions

A conditional expression **evaluates an expression based on a condition**. Conditional expressions have a completely different structure and do not include an explicit if. The syntax is shown below:

boolean-expression ? expression1 : expression2;

For example:

<pre>if (x > 0) y = 1; else y = -1;</pre>	==	<pre>y = x > 0 ? 1 : -1;</pre>
--	-----------	---

Loops



A loop can be used to tell a program to execute statements repeatedly.

Write a program to display the following message on the console 10 times .

"Welcome to programming!"

The while Loop



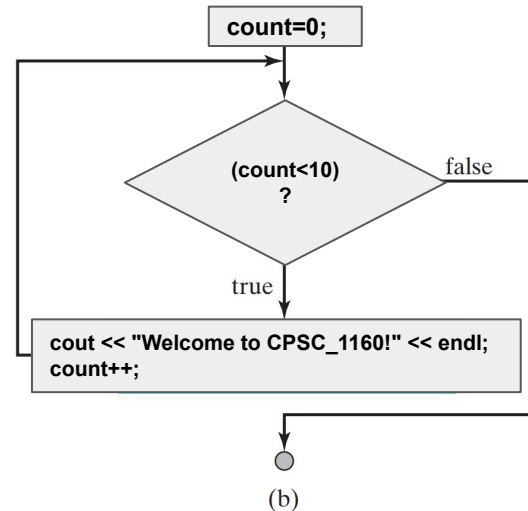
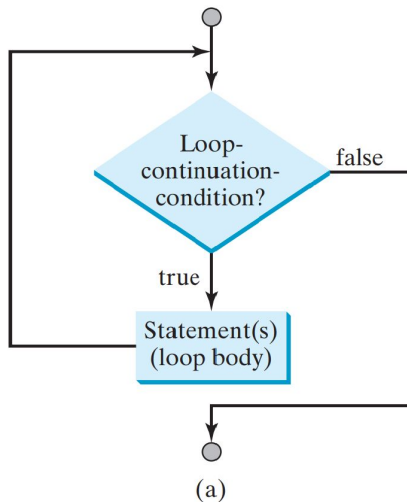
A while loop executes statements repeatedly while the condition is true.

The syntax for the while loop is

```
while (loop-continuation-condition)  
{  
  
    // Loop body  
  
    Statement(s);  
  
}
```

The while Loop

The while loop repeatedly executes the statements in the loop body when the loop-continuation-condition evaluates to true.



The while Loop

Part of the code is as follows:

```
int count = 0;

while (count < 10) // why < not <=
{
    printf ( "Welcome to programming");
    count++;
}
```

This type of loop is known as a **counter-controlled** loop.

The while Loop

What happens if the loop is written as follows?

```
int sum = 0, i = 1;
while (i < 10)
{
    sum = sum + i;
    i++;
}
```

VS

```
int sum = 0, i = 1;
while (i < 10)
{
    sum = sum + i;
}
```

The do-while Loop

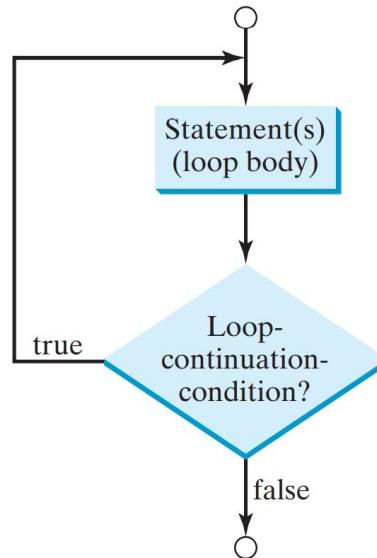
A do-while loop is the **same as a while loop except that it executes the loop body first and then checks** the loop continuation condition. Its syntax is as follows:

```
do
{
    // Loop body;

    Statement(s);
} while (loop-continuation-condition);
```

The do-while Loop

The do-while loop **executes the loop body first**, then checks the loop-continuation-condition to determine whether to continue or terminate the loop.



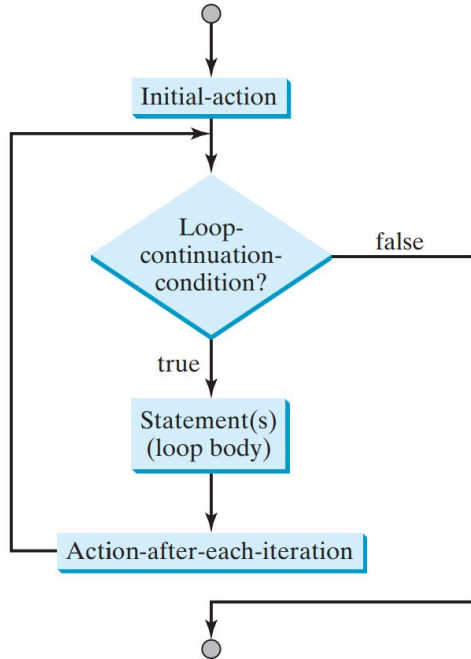
The for Loop

A for loop has a concise syntax for writing loops. In general, the syntax of a for loop is as shown below:

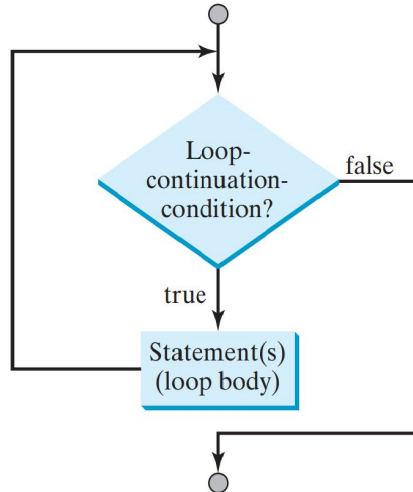
```
for (initial-action; loop-continuation-condition; action-after-each-iteration)
{
    // Loop body;
    Statement(s);
}
```

The for Loop

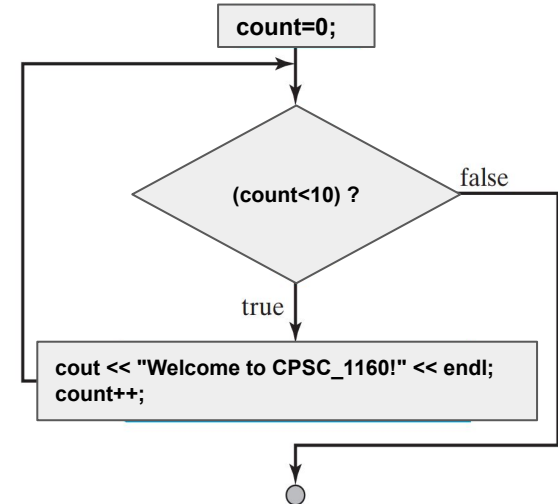
A for loop performs **an initial action once**, then repeatedly **executes the statements in the loop body**, and performs **an action after an iteration** when the loop-continuation-condition evaluates to true.



for loop



while loop



The for Loop

The loop control variable can be declared and initialized in the for loop. Here is an example:

```
for (int i = 0; i < 10; i++)  
{  
    printf ( "Welcome to programming");  
}
```

If there is **only one statement in the loop body**, as in this example, the **braces can be omitted**.

The for Loop

Do the following two loops result in the same value in sum?

```
for (int i = 0; i < 10; ++i)
{
    sum += i;
}
```

(a)

```
for (int i = 0; i < 10; i++)
{
    sum += i;
}
```

(b)

Which Loop to Use?



In general,

- a for loop may be used if **the number of repetitions is known in advance**;
- a while loop may be used if **the number of repetitions is not fixed**;
- a do-while loop can be used to replace a while loop if **the loop body has to be executed before the continuation condition is tested**.

Which Loop to Use?

```
while (loop-continuation-condition)
{
    // Loop body
}
```

(a)

Equivalent

```
for ( ; loop-continuation-condition; )
{
    // Loop body
}
```

(b)

```
for (initial-action;
     loop-continuation-condition;
     action-after-each-iteration)
{
    // Loop body;
}
```

(a)

Equivalent

```
initial-action;
while (loop-continuation-condition)
{
    // Loop body;
    action-after-each-iteration;
}
```

(b)

Nested Loops

Nested loops consist of an outer loop and one or more inner loops. Each time the outer loop is repeated, the inner loops are reentered, and started anew.

Be aware that a nested loop may take a long time to run. Consider the following loop nested in three levels:

```
for (int i = 0; i < 10000; i++)
```

```
    for (int j = 0; j < 10000; j++)
```

```
        for (int k = 0; k < 10000; k++)
```

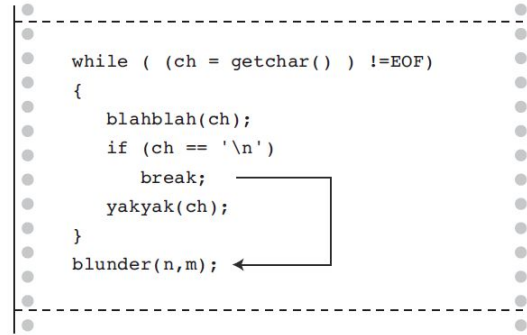
```
            statement(s); // {} will be needed for statements.
```

Keywords break and continue

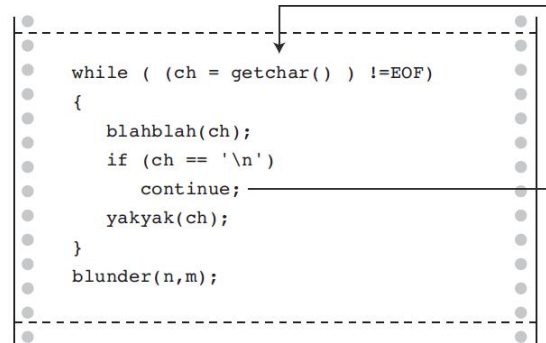
The **break** and **continue** keywords provide **additional controls in a loop**. The **continue** keyword breaks out of an iteration, while the **break** keyword breaks out of a loop.

- Use **break** in a loop to immediately terminate the loop.
- use **continue** in a loop to **end the current iteration**. Program control goes to the end of the loop body.

```
while ( (ch = getchar() ) != EOF)
{
    blahblah(ch);
    if (ch == '\n')
        break;
    yakyak(ch);
}
blunder(n,m);
```



```
while ( (ch = getchar() ) != EOF)
{
    blahblah(ch);
    if (ch == '\n')
        continue;
    yakyak(ch);
}
blunder(n,m);
```



Exercises

1. Use a **while** or **for** loop to compute the sum of the following cases:
 - a) 1, 2, 3, 4, ... 10
 - b) 1, 3, 5, 9
 - c) 1, 2, 3, 4, 6, 7, 8, 9, 10
2. Use nested **for** loops to display the following patterns separately:

***	#	*
***	##	*
***	###	*

Listing 7.12 The vowels.c Program

```
// vowels.c -- uses multiple labels
#include <stdio.h>
int main(void)
{
    char ch;
    int a_ct, e_ct, i_ct, o_ct, u_ct;

    a_ct = e_ct = i_ct = o_ct = u_ct = 0;

    printf("Enter some text; enter # to quit.\n");
    while ((ch = getchar()) != '#')
    {
        switch (ch)
        {
            case 'a' :
            case 'A' : a_ct++;
                      break;

            case 'e' :
            case 'E' : e_ct++;
                      break;

            case 'i' :
            case 'I' : i_ct++;
                      break;

            case 'o' :
            case 'O' : o_ct++;
                      break;

            case 'u' :
            case 'U' : u_ct++;
                      break;

            default : break;
        }
    }
    printf("number of vowels:  A   E   I   O   U\n");
    printf("                  %4d %4d %4d %4d %4d\n",
           a_ct, e_ct, i_ct, o_ct, u_ct);

    return 0;
}
```

Here's a sample run:

Enter some text; enter # to quit.

I see under the overseer.#

number of vowels:	A	E	I	O	U
	0	7	1	1	1

Reference



- **Stephen Prata. C Primer Plus (6th Edition). Addison-Wesley Professional, 2013.**